



**Hochschule Aalen
Technik und Wirtschaft**

Fragments of Silvaris

Ein 2D-RPG

Dokumentation

Spieleprogrammierung — Wintersemester 2025/2026

vorgelegt von

Aaron Seitz (3006614)

Robin Fitzek (3006199)

Burak Yigit (3005865)

Paul Weber (88487)

Februar 2026

Zusammenfassung

Fragments of Silvaris ist ein 2D Action-RPG, das in Unity 6 entwickelt wurde und sich am Spieldesign des beliebten MMORPGs NosTale orientiert. Das Projekt zeigt die Umsetzung verschiedener Game Development Konzepte, mit besonderem Fokus auf intelligentes Gegner-Verhalten und durchdachte Spielmechaniken.

Das Herzstück des Spiels ist ein raycasting-basiertes System, das Gegnern ermöglicht, selbstständig um Hindernisse zu navigieren. Statt in Wänden stecken zu bleiben, können die Gegner dank multi-direktionaler Raycasts und dynamischer Ausweichlogik intelligent auf Hindernisse reagieren.

Weitere Highlights sind ein arc-basiertes Kampfsystem mit 60°-Angriffskegel, ein umfassendes RPG-System mit Inventar, Ausrüstung und Levelprogressionen, sowie ein dynamisches Audio-System, das zwischen Kampf- und Friedensmusik wechselt. Das Spiel bietet 10 miteinander verbundene Szenen mit Boss-Begegnungen, ein Journal-System für die Story und eine polierte Benutzeroberfläche.

Das Projekt wurde von einem Team über einen Zeitraum von Oktober 2025 bis Februar 2026 entwickelt und erfolgreich umgesetzt.

Inhaltsverzeichnis

Abbildungsverzeichnis	6
Tabellenverzeichnis	7
1 Überblick	1
1.1 Spielkonzept	1
1.1.1 Spielgenre und Plattform	1
1.1.2 Zielgruppe und Setting	1
1.2 Projektziele	2
1.3 Entwicklerteam und Rollenverteilung	2
2 Gameplay und Mechaniken	3
2.1 Kern-Spielmechaniken	3
2.1.1 Bewegungssteuerung	3
2.1.2 Kampfsystem	3
2.1.3 Schadenssystem und Verteidigung	4
2.2 Progression und Level-Design	4
2.2.1 Erfahrungssystem	4
2.2.2 Statistik-System	4
2.2.3 Level-Design und Weltenaufbau	5
3 Story und Narrative	7
3.1 Hintergrundgeschichte	7
3.2 Charaktere	7
3.2.1 Spieler-Charakter	7
3.2.2 Nicht-Spieler-Charaktere (NPCs)	7
3.2.3 Gegnerische Charaktere	8
3.3 Journal-System	9
4 Grafik und Sound	10
4.1 Visueller Stil	10
4.1.1 Art-Style	10
4.1.2 Pixel-Perfect Rendering	10
4.1.3 Farbschema	10
4.2 Benutzeroberfläche	10
4.2.1 Minimap und Kartenansicht	11
4.2.2 Shop- und Händler-Interface	12
4.3 Sound-Design	12
4.3.1 Audio-Management	12
4.3.2 Musik-System	12
4.3.3 Sound Effects	13
5 Gegner-Verhalten und Steuerung	14
5.1 Überblick der Gegner-Systeme	14

5.2	Enemy-Bewegungs-KI	14
5.2.1	State Machine Architektur	14
5.2.2	Idle State – Wandering Behavior	15
5.2.3	Raycasting-Based Obstacle Avoidance	15
5.2.4	Chasing State – Player Pursuit	17
5.3	Angriffs-Logik	18
5.3.1	Attack Telegraph System	18
5.3.2	Contact Damage	19
5.4	Weitere KI-Features	19
5.4.1	NPC Wandering	19
5.4.2	Boss Wave System	20
6	Technische Implementierung	21
6.1	Game Engine und Tools	21
6.1.1	Unity Engine	21
6.1.2	Universal Render Pipeline (URP)	21
6.1.3	Key Packages	21
6.2	Code-Architektur	21
6.2.1	Singleton Pattern	21
6.2.2	Event-Driven Architecture	22
6.2.3	Inventory System (MVC Pattern)	23
6.2.4	Scene Management	23
6.2.5	Save/Load System	24
6.3	Performance-Optimierungen	24
6.3.1	Culling-System	24
6.3.2	Animator Culling	24
6.3.3	Dynamic Quality	24
7	Entwicklungsprozess	25
7.1	Planung und Projektmanagement	25
7.1.1	Entwicklungsphasen	25
7.1.2	Versionskontrolle	26
7.2	Meilensteine und Deliverables	26
7.3	Testing und Quality Assurance	26
7.3.1	Testing-Strategien	26
7.3.2	Bug Tracking	26
7.4	Herausforderungen und Lösungen	27
7.4.1	Challenge 1: Enemy Pathfinding	27
7.4.2	Challenge 2: Scene Persistence	27
7.4.3	Challenge 3: Combat Feel	27
7.4.4	Challenge 4: Inventory Serialisierung	27
7.4.5	Challenge 5: Defense Scaling	28
7.5	Design-Entscheidungen	28
7.5.1	Top-Down vs. Side-Scroller	28
7.5.2	Combat System: Target Lock vs. Free Aim	28
7.5.3	Gegnersteuerung: Raycasting vs. A* Pathfinding	28
7.5.4	Save System: Local vs. Cloud	29
8	Fazit	30
8.1	Zusammenfassung der Ergebnisse	30
8.2	Erreichte Ziele	30

8.3	Reflexion über Projektziele	30
8.3.1	Was gut lief	30
8.3.2	Was verbessert werden könnte	31
8.4	Probleme und Lösungswege	31
8.5	Zukünftige Erweiterungsmöglichkeiten	31
8.5.1	Technische Enhancements	31
8.5.2	Content Expansionen	31
8.5.3	Quality of Life	31
8.6	Persönliche Erkenntnisse und Learnings	32
8.6.1	Technische Insights	32
8.6.2	Game Design Insights	32
8.6.3	Team-Erkenntnisse	32
8.7	Schlusswort	32
9	Verwendete Assets und Lizenzen	33
9.1	Grafik-Assets	33
9.1.1	Pixel Art Potion Pack (Animated)	33
9.1.2	Farm RPG - Character and Environment Sprites	33
9.1.3	Kittens & Cats Character 32x32 Pixel Animation Pack	33
9.2	Audio-Assets	34
9.2.1	Musik	34
9.2.2	Sound Effects	34
9.3	Lizenzdokumente	34

Abbildungsverzeichnis

1.1	Hauptmenü des Spiels	1
2.1	Ambiente-Elemente: Kühe, Raben und Vegetation	5
2.2	Findbare Chest (Truhe) im Spiel	6
3.1	Charakter-Auswahlbildschirm	7
3.2	NPCs in einem Gebäude	8
3.3	Wandernde NPCs im Freien	8
3.4	Slime-Gegner mit Healthbars und Level-Anzeige	9
4.1	Pause-Menü mit Audio-Einstellungen	11
4.2	Minimap-Ansicht eines Dorfgebiets	11
4.3	Gezoomte Kartenansicht mit detaillierter Übersicht	12
4.4	Shop-Interface mit verfügbaren Items	12

Tabellenverzeichnis

5.1	Gegner-Zustandsübergänge	14
5.2	Obstacle Avoidance Logik nach Raycast-Ergebnissen	17
5.3	Attack Telegraph Parameter	18
6.1	Unity Packages und Versionen	21

Abkürzungsverzeichnis

RPG Role-Playing Game

NPC Non-Player Character

UI User Interface

URP Universal Render Pipeline

XP Experience Points

HP Health Points

FPS Frames Per Second

FSM Finite State Machine

SFX Sound Effects

MVP Minimum Viable Product

A* A* Pathfinding-Algorithmus

MVC Model-View-Controller

1 Überblick

1.1 Spielkonzept

Fragments of Silvaris ist ein 2D Action-RPG mit Top-Down-Perspektive, entwickelt als Abschlussprojekt für die Vorlesung Spieleprogrammierung an der Hochschule Aalen. Das Spiel orientiert sich thematisch und mechanisch am beliebten MMORPG NoTale **spieleprogrammierung'course** kombiniert aber klassische 2D-RPG-Mechaniken mit modernen Game Development Standards.



Abbildung 1.1: Hauptmenü des Spiels

1.1.1 Spielgenre und Plattform

Das Spiel wird als 2D Action-RPG kategorisiert und auf dem PC (Windows/Linux) ausgeführt. Es nutzt die moderne Unity 6 Engine **unity'engine** mit dem Universal Render Pipeline (URP) **unity'urp'2024** für optimierte Grafik-Rendering. Das Spiel läuft in Echtzeit und erfordert eine Framerate von mindestens 60 FPS für optimales Spielerlebnis.

1.1.2 Zielgruppe und Setting

Die Zielgruppe sind Spieler im Alter von 16+, die sich für klassische 2D Action-RPGs interessieren. Das Spiel spielt in einer Fantasy-Welt, in der mysteriöse Slime-Kreaturen eine Invasion führen. Der Spieler übernimmt die Rolle eines Helden, der die Aufgabe hat, diese Bedrohung zu bekämpfen.

1.2 Projektziele

Das Projektziel war die Implementierung eines funktionsfähigen 2D-RPGs mit folgenden Kernzielen:

1. Implementierung eines ausgefeilten Gegner-Verhaltens mit Raycasting-basiertem Obstacle Avoidance
2. Entwicklung eines engaging Kampfsystems mit Arc-basierte Treffererkennung
3. Erstellung eines umfassenden RPG-Systems mit Inventar, Ausrüstung und Progression
4. Integration von dynamischem Audio und polierter Benutzeroberfläche
5. Dokumentation des gesamten Entwicklungsprozesses und technischer Entscheidungen

1.3 Entwicklerteam und Rollenverteilung

Das Projekt wurde von einem Team bestehend aus vier Entwicklern umgesetzt:

- **Robin Fitzek (3006199)**: 5 Scenes/Maps Design, Spiellogik und UI, Save & Load System, Story Komponenten
- **Burak Yigit (3005865)**: Spiellogik, Shop System, 3 Maps/Scenes, Gold System
- **Aaron Seitz (3006614)**: Game Systems, Architecture, Advanced NPCs, Enemies, Player Gameplay, Inventory
- **Paul Weber (88487)**: NPC Systems, Ice Map Development

Entwicklungszeitraum: Oktober 2025 – Februar 2026

2 Gameplay und Mechaniken

2.1 Kern-Spielmechaniken

2.1.1 Bewegungssteuerung

Das Spiel unterstützt 8-direktionale Bewegung über WASD-Tasten. Der Spieler kann sich in folgende Richtungen bewegen: oben, unten, links, rechts und diagonale Richtungen (oben-links, oben-rechts, unten-links, unten-rechts). Die Animation blended zwischen den vier kardinal Richtungen basierend auf der Eingabe.

Die Bewegungsgeschwindigkeit ist konfigurierbar und wird durch Statuseffekte (Potions) beeinflusst. Der Charakter wird automatisch in die Bewegungsrichtung gedreht.

2.1.2 Kampfsystem

Das Kampfsystem basiert auf Arc-basierten Melee-Attacken, die der Spieler mit der Maus steuert. Die Attacke wird in der Richtung vom Spieler zur Mausposition durchgeführt.

- **Attackenreichweite:** 2.0 Units
- **Angriffswinkel:** 60° Kegel (30° links + 30° rechts von der Angriffsrichtung)
- **Attackengeschwindigkeit:** 2 Attacken pro Sekunde (50ms Cooldown)
- **Multi-Target:** Kann mehrere Gegner gleichzeitig treffen

Die Treffererkennung funktioniert durch eine Kombination aus Distanzcheck und Winkelcheck:

```
1 void DetectAndDamageEnemiesInArc(Vector2 origin, Vector2
   attackDirection)
2 {
3     // Alle Gegner im maximalen Radius finden
4     Collider2D[] potentialTargets = Physics2D.OverlapCircleAll(
5         origin, attackRange, enemyLayers);
6
7     foreach (Collider2D target in potentialTargets)
8     {
9         Vector2 toEnemy = (Vector2)target.transform.position - origin;
10        float angleToEnemy = Vector2.Angle(attackDirection, toEnemy.
normalized);
11
12        // Gegner nur treffen wenn im Kegel
13        if (angleToEnemy <= attackAngle / 2f)
14        {
15            target.GetComponent<EnemyHealth>()?.TakeDamage(
attackDamage);
16        }
17    }
```

Listing 2.1: Arc-basierte Treffererkennung (PlayerCombat.cs)

2.1.3 Schadenssystem und Verteidigung

Das Schadenssystem implementiert eine Defense-Formel mit abnehmenden Erträgen, um zu verhindern, dass Spieler auf hohen Levels invincibel werden:

```
1 float K = 20f; // Balancing-Konstante
2 float reduction = totalDefense / (totalDefense + K);
3 reduction = Mathf.Min(reduction, 0.8f); // Maximum 80% Reduktion
4 float damageAfterDef = Mathf.Max(damage * (1f - reduction), 1f);
```

Listing 2.2: Defense-Formel mit Diminishing Returns

Diese Formel stellt sicher, dass:

- Erhöhte Verteidigung immer Vorteile bringt
- Die Steigerung aber abnehmend wird (Diminishing Returns)
- Spieler niemals mehr als 80
- Minimum 1 Schaden pro Hit garantiert ist

2.2 Progression und Level-Design

2.2.1 Erfahrungssystem

Das Spiel nutzt ein exponentielles Level-Progressionssystem. Der Spieler erhält Erfahrungspunkte (XP) durch:

- Besiegen von Gegnern (20 XP Base + Level-Bonus)
- Abschluss von Boss-Encounters

Die erforderliche XP für den nächsten Level wird exponentiell skaliert:

$$XP_{\text{required}} = XP_{\text{previous}} \times 1.5 \quad (2.1)$$

2.2.2 Statistik-System

Spieler-Charaktere haben folgende Statistiken:

- **Gesundheit (HP):** $100 + (\text{Level} \times 10)$
- **Mana (MP):** $50 + (\text{Level} \times 5)$
- **Schaden (DMG):** $10 + (\text{Level} \times 2)$
- **Verteidigung (DEF):** Addiert von Ausrüstung
- **Kritische Quote:** Prozentsatz für Kritische Treffer
- **Kritischer Schaden:** Multiplikator für Kritische Treffer

Gegner skalieren automatisch zum Spieler-Level, um ein ausgeglichenes Kampferlebnis zu bieten.

2.2.3 Level-Design und Weltenaufbau

Das Spiel besteht aus 10 miteinander verbundenen Szenen:

- 1 Hauptmenü-Szene
- 1 Basis-Szene (Heimat-Basis)
- 3 Dorf-Szenen mit NPC-Interaktionen
- 3 Dungeon/Wildnis-Szenen mit Gegnern
- 2 Boss-Arena-Szenen

Szenen sind über Trigger-Zonen miteinander verbunden und implementieren ein Spawn-Point-System für nahtlose Übergänge.

Ambiente-Elemente und Sammelobjekte

Die Spielwelt enthält verschiedene Ambiente-Elemente und Sammelobjekte, die zur Immersion und Spielmechanik beitragen:

- **Tiere:** Kühe und Raben bevölkern die Landschaft und schaffen eine lebendige Spielwelt
- **Chests:** Versteckte Truhen in verschiedenen Szenen enthalten Story-Fragmente, die sich zu einem größeren narrativen Puzzle zusammensetzen. Das Finden aller Chests enthüllt die vollständige Hintergrundgeschichte der Welt
- **Vegetation:** Bäume, Blumen und Gras sorgen für visuelles Detail



Abbildung 2.1: Ambiente-Elemente: Kühe, Raben und Vegetation



Abbildung 2.2: Findbare Chest (Truhe) im Spiel

3 Story und Narrative

3.1 Hintergrundgeschichte

In einer Fantasy-Welt existiert eine alte Zivilisation mit großer magischer Macht. Durch ein mysteriöses Ritual werden slimeähnliche Kreaturen beschworen, die beginnen, die Welt zu infiltrieren. Der Spieler wird von den verbliebenen Bewohnern als Held herbeigerufen, um diese Invasion zu stoppen.

Die Geschichte wird nicht durch Zwischensequenzen, sondern durch das Journal-System erzählt.

3.2 Charaktere

3.2.1 Spieler-Charakter

Der Spieler kontrolliert einen Abenteurer, der die Mission hat, die Slime-Invasion zu stoppen. Der Charakter hat fünf verschiedene Erscheinungsformen zur Auswahl, was eine gewisse Personalisierung ermöglicht.

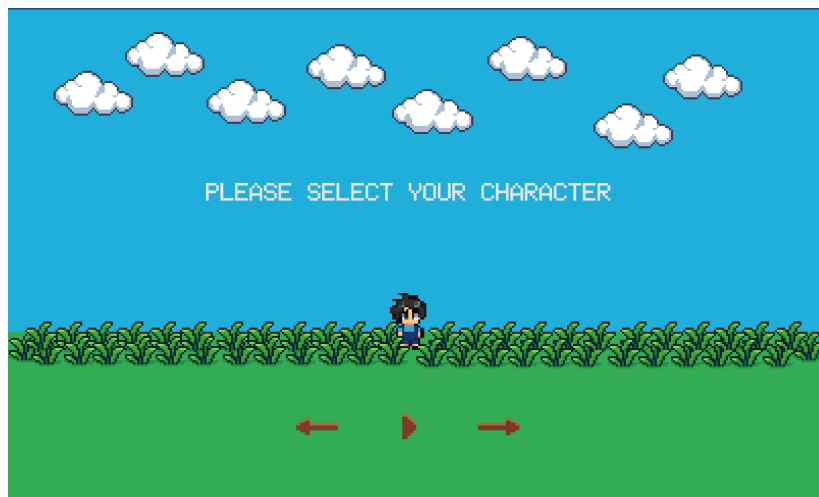


Abbildung 3.1: Charakter-Auswahlbildschirm

3.2.2 Nicht-Spieler-Charaktere (NPCs)

- **Wandernde NPCs:** Freundliche Charaktere, die in den Dörfern herumlaufen und mit dem Spieler interagieren können
- **Kaufleute:** NPCs die Items verkaufen
- **Händler:** Spezialisierte NPCs für bestimmte Güter



Abbildung 3.2: NPCs in einem Gebäude



Abbildung 3.3: Wandernde NPCs im Freien

3.2.3 Gegnerische Charaktere

Das Spiel enthält vier Slime-Varianten als Gegner:

- **Blue Slime:** Defensiv, höhere Verteidigung
- **Green Slime:** Standard-Gegner
- **Red Slime:** Aggressiv, höherer Schaden
- **Yellow Slime:** Schnell, höhere Geschwindigkeit

Boss-Varianten dieser Slimes spawnen mit höherem Level in Boss-Encounters.

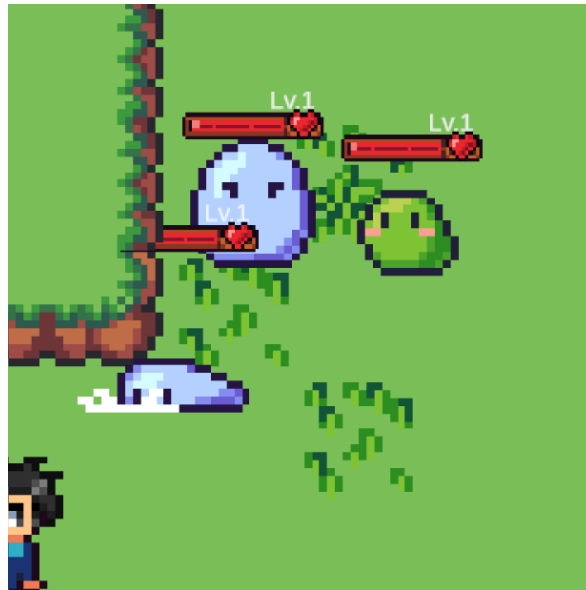


Abbildung 3.4: Slime-Gegner mit Healthbars und Level-Anzeige

3.3 Journal-System

Das Spiel implementiert ein Journal-System, das die Geschichte erzählt und dem Spieler Hinweise gibt. Journal-Einträge werden freigeschaltet durch:

- Erreichen bestimmter Szenen
- Besiegen bestimmter Anzahl von Gegnern
- Erreichen bestimmter Level-Milestones
- Abschluss von Boss-Encounters

Das Journal enthält über 12 Einträge und wird über eine UI-Overlay-Schnittstelle zugegriffen.

4 Grafik und Sound

4.1 Visueller Stil

4.1.1 Art-Style

Das Spiel nutzt einen 2D Pixel-Art-Stil, inspiriert von klassischen RPGs wie *NosTale*, *Terraria* und *The Legend of Zelda*. Der Stilmix betont Gameplay gegenüber realistischer Grafik.

4.1.2 Pixel-Perfect Rendering

Das Spiel nutzt Unitys Pixel Perfect Camera Package **unity'2d'pixel'perfect**, um sicherzustellen, dass Sprites pixelgenau ohne anti-aliasing gerendert werden. Dies ist essentiell für die Aufrechterhaltung der visuellen Konsistenz des Pixel-Art-Stils.

4.1.3 Farbschema

Das Farbschema variiert je nach Biom:

- **Grünland:** Grüne Töne, warme Erdfarben
- **Dorf:** Warm und einladend
- **Dungeon:** Düster, mit purpurroten Tönen
- **Boss-Arena:** Dramatisch, kontrastig

4.2 Benutzeroberfläche

Die UI ist funktional und wird mit TextMesh Pro **unity'textmesh'pro** gerendert. Zu den UI-Elementen gehören:

- **Health/Mana/Stamina Bars:** Oben links, zeigt aktuelle und maximale Werte
- **Inventory System:** 49-Slot Grid (7x7) für Items
- **Equipment Screen:** 6 Ausrüstungsplätze (Waffe, Helm, Rüstung, Stiefel, Handschuhe, Accessoire)
- **Hotbar:** Schnellzugriff auf 10 Items
- **Minimap:** Kleine Karte mit Spieler-Position
- **Stats Sheet:** Zeigt alle Charakterstatistiken
- **Pause Menu:** Einstellungen, Volumenregler, Quit
- **Journal:** Zeigt Story-Einträge



Abbildung 4.1: Pause-Menü mit Audio-Einstellungen

4.2.1 Minimap und Kartenansicht

Die Minimap zeigt dem Spieler seine Position in der aktuellen Szene. Zusätzlich gibt es eine Zoom-Funktion über die Taste **M**, die eine detaillierte Übersicht der gesamten Karte bietet und dem Spieler bei der Orientierung hilft.



Abbildung 4.2: Minimap-Ansicht eines Dorfgebiets

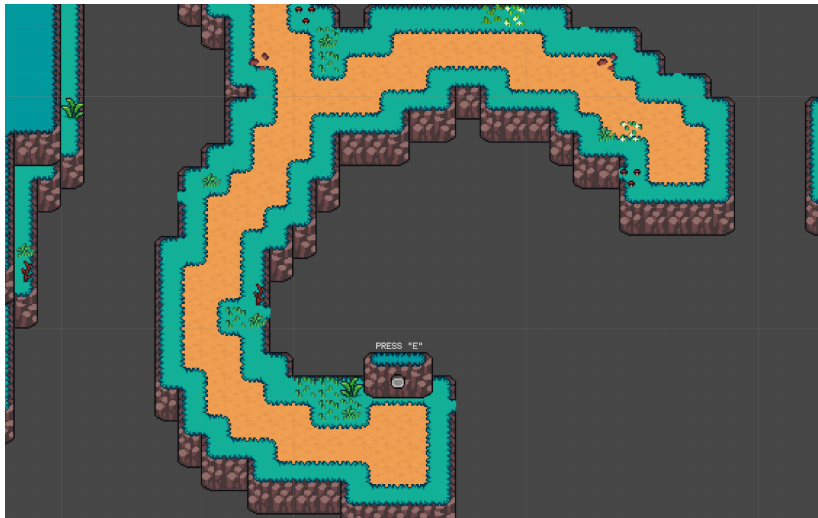


Abbildung 4.3: Gezoomte Kartenansicht mit detaillierter Übersicht

4.2.2 Shop- und Händler-Interface

Das Spiel enthält ein Shop-System, über das Spieler Items kaufen und verkaufen können. Die Shop-UI zeigt verfügbare Items mit Icons und ermöglicht direkte Kaufinteraktionen.



Abbildung 4.4: Shop-Interface mit verfügbaren Items

4.3 Sound-Design

4.3.1 Audio-Management

Das Audio-System wird über einen zentralen AudioManager Singleton **unity engine** verwaltet, der mit dem DontDestroyOnLoad-Pattern szenenweit persistent ist.

4.3.2 Musik-System

Das Spiel implementiert ein dynamisches Musik-System mit zwei Zuständen:

- **Peaceful Music:** Standard-Hintergrundmusik für Exploration
- **Combat Music:** Intensivere Musik wenn Gegner in Nähe sind

Das System transistioniert automatisch zwischen diesen Zuständen mit einem 1-Sekunden-Crossfade. Wenn kein Kampf für 2 Sekunden stattgefunden hat, kehrt die Musik zu Peaceful zurück.

```

1 public void EnterCombat()
2 {
3     lastCombatTime = Time.time;
4     if (!isInCombat)
5     {
6         isInCombat = true;
7         StartCoroutine(CrossfadeMusic(combatMusic, 1f));
8         StartCoroutine(CheckCombatEnd());
9     }
10 }
11
12 private IEnumerator CheckCombatEnd()
13 {
14     while (isInCombat)
15     {
16         if (Time.time - lastCombatTime > combatMusicDelay)
17             ExitCombat();
18         yield return new WaitForSeconds(0.5f);
19     }
20 }

```

Listing 4.1: Combat Music State Machine (AudioManager.cs)

4.3.3 Sound Effects

Die Soundeffekte sind kategorisiert nach:

- **Battle SFX:** Slash (Angriff), Hit (Treffer), Miss (Verfehlt), Gegner-Tod
- **UI SFX:** Equipment, Buy, Hover, Confirm, Deny
- **Shop SFX:** Shop-Open, Item-Sold, Insufficient-Gold
- **Movement SFX:** Footsteps (terrain-abhängig), Jump, Landing, Teleport

5 Gegner-Verhalten und Steuerung

Dies ist das Kernkapitel der Dokumentation, fokussiert auf das Raycasting-basierte Gegner-Verhalten.

5.1 Überblick der Gegner-Systeme

Das KI-System des Spiels besteht aus mehreren Komponenten:

1. **State Machine:** 3-Zustands-Finite-State-Machine für Enemy-Verhalten
2. **Obstacle Avoidance:** Raycasting-basiertes System zur Hindernisnavigation
3. **Player Detection:** Proximity-basierter Erkennungsmechanismus
4. **Combat Logic:** Angriffs-Entscheidungen mit Telegraph-System

5.2 Enemy-Bewegungs-KI

5.2.1 State Machine Architektur

Die Enemy-KI verwendet eine 3-Zustands-Zustandsmaschine:

```
1 public enum EnemyState
2 {
3     Idle,           // Wandering, Suche nach Spieler
4     Chasing,        // Verfolgung des Spielers
5     Attacking       // Angriff auf den Spieler
6 }
```

Listing 5.1: Enemy State Machine Enum

Die Zustandsübergänge sind wie folgt definiert:

Tabelle 5.1: Gegner-Zustandsübergänge

Von	Nach	Bedingung
Idle	Chasing	$\text{distanceToPlayer} \leq 5.0$
Chasing	Attacking	$\text{distanceToPlayer} \leq 2.0$
Attacking	Chasing	$\text{distanceToPlayer} > 4.0$
Chasing	Idle	$\text{distanceToPlayer} > 5.0$

5.2.2 Idle State – Wandering Behavior

Wenn kein Spieler erkannt wird, bewegt sich der Gegner in einem Wander-Muster. Das Wandering-Verhalten ist wie folgt implementiert:

1. Zufälliger Ziel-Punkt wird innerhalb eines Radius um die Spawn-Position ausgewählt (standardmäßig 3 Units)
2. Der Gegner bewegt sich zum Ziel mit der Wander-Geschwindigkeit (1 Unit/s)
3. Bei Erreichen des Ziels: Pause für random Dauer (1-4 Sekunden)
4. Zurück zu Schritt 1

```
1 void HandleWandering()
2 {
3     if (wanderPauseTimer > 0)
4     {
5         wanderPauseTimer -= Time.deltaTime;
6         rb.linearVelocity = Vector2.zero;
7         return;
8     }
9
10    if (!isWandering)
11    {
12        Vector2 randomOffset = Random.insideUnitCircle * wanderRadius;
13        wanderTarget = spawnPosition + randomOffset;
14        isWandering = true;
15    }
16
17    Vector2 direction = (wanderTarget - (Vector2)transform.position).
normalized;
18    direction = ApplyObstacleAvoidance(direction);
19    rb.linearVelocity = direction * wanderSpeed;
20 }
```

Listing 5.2: Wandering Behavior (Enemy_Movement.cs)

Das Wandering-Verhalten nutzt auch das Obstacle Avoidance System (siehe Abschnitt 5.2.3), um Hindernisse zu vermeiden.

5.2.3 Raycasting-Based Obstacle Avoidance

Motivation und Problemstellung

Die ursprüngliche Implementierung der Enemy-Bewegung verwendete eine einfache direkte Verfolgung des Spielers. Dies führte zu folgendem Problem:

Gegner würden in Wänden und anderen Hindernissen steckenbleiben, wenn der Spieler auf der anderen Seite eines Hindernisses war. Dies führte zu schlechtem Gameplay, da Gegner nicht intelligent um Hindernisse navigieren konnten.

Das Obstacle Avoidance System löst dieses Problem durch Multi-direktionales Raycasting, das es Gegnern ermöglicht, Hindernisse intelligent zu umgehen.

Technische Implementierung

Das System nutzt ein 3-Ray-Raycasting-System:

1. **Forward Ray:** Hauptrichtung der beabsichtigten Bewegung

- Länge: 1.5 Units (raycastDistance)
- Winkel: 0° (direkt nach vorne)

2. **Left Ray:** 30° nach links rotiert

- Länge: $0.7 \times \text{raycastDistance} = 1.05$ Units
- Winkel: 30° (nach links)

3. **Right Ray:** 30° nach rechts rotiert

- Länge: $0.7 \times \text{raycastDistance} = 1.05$ Units
- Winkel: -30° (nach rechts)

Alle Raycasts werden gegen den NPC.Collision Layer durchgeführt, der Wände und andere Hindernisse enthält (aber nicht den Spieler).

Obstacle Avoidance Algorithmus

Die Ausweichlogik basiert auf welche Raycasts ein Hindernis treffen:

```
1 Vector2 ApplyObstacleAvoidance(Vector2 desiredDirection)
2 {
3     if (obstacleLayer == 0) return desiredDirection;
4
5     // Forward Raycast
6     RaycastHit2D hit = Physics2D.Raycast(transform.position,
7         desiredDirection, raycastDistance, obstacleLayer);
8
9     if (hit.collider != null)
10    {
11        // Hindernis erkannt! Berechne Ausweichrichtung
12        Vector2 avoidDirection = CalculateAvoidanceDirection(
13            desiredDirection, hit.normal);
14        return (desiredDirection + avoidDirection * avoidanceStrength)
15            .normalized;
16    }
17
18    // Seitliche Raycasts
19    Vector2 leftRay = RotateVector(desiredDirection, 30f);
20    Vector2 rightRay = RotateVector(desiredDirection, -30f);
21
22    RaycastHit2D hitLeft = Physics2D.Raycast(transform.position,
23        leftRay, raycastDistance * 0.7f, obstacleLayer);
24    RaycastHit2D hitRight = Physics2D.Raycast(transform.position,
25        rightRay, raycastDistance * 0.7f, obstacleLayer);
26
27    if (hitLeft.collider != null && hitRight.collider == null)
28        return (desiredDirection + RotateVector(desiredDirection, -45f)
29            * avoidanceStrength * 0.5f).normalized;
```

```

30     else if (hitRight.collider != null && hitLeft.collider == null)
31         return (desiredDirection + RotateVector(desiredDirection, 45f)
32             *
33                 avoidanceStrength * 0.5f).normalized;
34     else if (hitLeft.collider != null && hitRight.collider != null)
35         return -desiredDirection;
36
37     return desiredDirection;
38 }

```

Listing 5.3: Obstacle Avoidance Algorithmus (Enemy_Movement.cs)

Die Ausweichlogik funktioniert nach diesem Muster:

Tabelle 5.2: Obstacle Avoidance Logik nach Raycast-Ergebnissen

Raycast-Treffer	Aktion	Stärke
Forward Only	Seitlich ausweichen (perpendicular zur Normale)	1.0x
Left Only, Right Free	Nach rechts ausweichen (-45°)	0.5x
Right Only, Left Free	Nach links ausweichen (+45°)	0.5x
Both Left & Right	Umdrehen (-180°)	1.0x
None	Keine Ausweichung	–

Smoothing und Performance

Um jittery Movement zu vermeiden, implementiert das System:

- **Cooldown System:** 0.2 Sekunden Cooldown zwischen Ausweichentscheidungen
- **Direction Caching:** Die letzte Ausweichrichtung wird cached für sanfte Übergänge
- **Weighted Blending:** Gewichtete Kombination von Bewegungs- und Ausweichrichtung

Die Performance-Auswirkung von Raycasting ist minimal, da nur 3 Raycasts pro Frame durchgeführt werden (mit 0.2s Cooldown reduziert sich dies auf etwa 15 Raycasts pro Sekunde pro Enemy).

5.2.4 Chasing State – Player Pursuit

Wenn der Spieler erkannt wird ($\text{Distanz} \leq \text{detectionRange}$), betritt der Gegner den Chasing State. In diesem State:

1. Die Richtung zum Spieler wird jedes Frame berechnet
2. Das Obstacle Avoidance System wird angewendet
3. Der Gegner bewegt sich mit speed (2 Units/s) in diese Richtung
4. Der Gegner wendet sich dem Spieler zu (Sprite-Flipping basierend auf Richtung)
5. Audio-Manager wird informiert (Combat Music startet)

5.3 Angriffs-Logik

5.3.1 Attack Telegraph System

Ein zentrales Feature der Combat-Mechanik ist das Attack Telegraph System. Dies ermöglicht es dem Spieler, Gegner-Angriffe zu antizipieren und auszuweichen.

Motivation

In frühen Spielversionen führten Gegner Angriffe ladezeit durch. Dies führte zu Spieler-Frustration, da keine Reaktionsmöglichkeit existierte. Das Telegraph System löst dies durch eine "Vorwarnungszeit" vor dem eigentlichen Schaden.

Parameter

Tabelle 5.3: Attack Telegraph Parameter

Parameter	Wert	Beschreibung
attackRange	2.0 Units	Entfernung für Angriffsentscheidung
attackHitRange	1.0 Units	Tatsächliche Trefferreichweite
attackWindupTime	0.4 Sekunden	Telegraph-Zeit
attackCooldown	1.5 Sekunden	Cooldown zwischen Angriffen

Beachte, dass `attackRange > attackHitRange`. Dies bedeutet:

- Gegner beginnen Angriff bei 2.0 Units Distanz
- Aber Schaden wird nur angewendet wenn Spieler bei 1.0 Units ist
- Dies ermöglicht Spieler, während des Wind-ups auszuweichen

Attack Sequence

Eine Angriff läuft wie folgt ab:

1. Gegner-Distanz $\leq$ attackRange $\rightarrow$ Attacking State
2. Attack Animation wird gestartet
3. Wind-up Zeit (0.4s) – **Spieler kann ausweichen!**
4. Nach Wind-up: Prüfe ob Spieler noch in attackHitRange
5. IF YES: Schaden anwenden — IF NO: Angriff verfehlt
6. Cooldown warten (1.5s)
7. Zurück zu Schritt 1

```

1 IEnumerator AttackWithWindup()
2 {
3     isAttacking = true;
4     anim.SetTrigger("Attack");
5
6     // Wind-up Zeit - Spieler kann ausweichen!
7     yield return new WaitForSeconds(attackWindupTime);
8
9     // NOW: Check if player still in range
10    if (player != null)
11    {
12        float currentDistance = Vector2.Distance(
13            transform.position, player.position);
14
15        if (currentDistance <= attackHitRange)
16        {
17            PlayerStats playerStats = player
18                .GetComponent<PlayerStats>();
19            if (playerStats != null)
20                playerStats.TakeDamage(attackDamage);
21        }
22    }
23
24    yield return new WaitForSeconds(0.3f);
25    isAttacking = false;
26 }

```

Listing 5.4: Attack mit Wind-up (Enemy_Movement.cs)

5.3.2 Contact Damage

Zusätzlich zu Angriff-basierten Schaden können Gegner auch durch Kontakt Schaden verursachen (separate Collision-basierte Schaden). Dies ist implementiert über einen OnCollisionStay-Callback, der regelmäßig (etwa 1x pro Sekunde) Schaden anwendet.

5.4 Weitere KI-Features

5.4.1 NPC Wandering

Freundliche NPCs nutzen ein ähnliches Wandering-System wie Gegner im Idle State. Allerdings haben sie unterschiedliche Konfigurationen:

- Größere wanderRadius (bis zu 10 Units)
- Längere Pausen zwischen Bewegungen
- Keine Raycasting-Hinderniserkennung (einfacher)

5.4.2 Boss Wave System

Boss-Encounters verwenden ein Wave-basiertes System (SlimeWaveController). Charakteristiken:

- Ziel: 60 Slimes besiegen
- Spieler kontrolliert Spawn-Rate über Buttons
- Automatisches Fallback: 2 Slimes alle 60 Sekunden (wenn Spieler nichts macht)
- Boss-Slimes spawnen mit `playerLevel + 3` für Herausforderung
- Journal-Einträge entsperren bei 20, 40, 60 Kills

6 Technische Implementierung

6.1 Game Engine und Tools

6.1.1 Unity Engine

Das Spiel wurde in **Unity 6000.2.7f2 unity'engine** entwickelt, der neuesten LTS-Version von Unity. Die Wahl von Unity war strategisch:

- Für 2D-Spiele optimiert
- Schnelle Iteration und Playtesting
- Großer Asset Store für vorgefertigte Komponenten
- Starke C# Integration mit modernem Tooling

6.1.2 Universal Render Pipeline (URP)

Das Projekt nutzt Universal Render Pipeline (URP) 17.2.0 **unity'urp'2024** für Grafik-Rendering. URP ist ein skriptbarer Rendering-Pipeline, der Flexibilität und Performance bietet.

6.1.3 Key Packages

Folgende wichtige Packages wurden integriert:

Tabelle 6.1: Unity Packages und Versionen

Package	Version	Verwendung
2D Animation	12.0.2	Sprite-Rigging und Skelettanimation
Pixel Perfect	5.1.1	Pixel-perfektes Rendering
Input System	1.14.2	Moderne Input-Verarbeitung
Cinemachine	3.1.4	Kamera-Kontrolle

6.2 Code-Architektur

6.2.1 Singleton Pattern

Das Spiel nutzt das Singleton Pattern für Manager-Klassen, die global zugegriffen werden müssen:

- **AudioManager**: Zentrale Audio-Verwaltung
- **SaveManager**: Speichern/Laden von Spielzustand

- **SceneManager**: Szenen-Übergänge mit Spawn Points
- **GoldManager**: Währungs-Management
- **NotificationManager**: Toast-Benachrichtigungen

Das Singleton Pattern wird mit dem DontDestroyOnLoad-Ansatz implementiert:

```

1 public class AudioManager : MonoBehaviour
2 {
3     public static AudioManager Instance { get; private set; }
4
5     private void Awake()
6     {
7         if (Instance == null)
8         {
9             Instance = this;
10            DontDestroyOnLoad(gameObject);
11        }
12        else
13        {
14            Destroy(gameObject);
15        }
16    }
17 }

```

Listing 6.1: Singleton Pattern Implementierung

Vorteile:

- Globaler Zugriff auf wichtige Manager
- Persistenz über Szenen-Grenzen hinweg
- Verhindert Duplikate durch Pattern-Implementierung

6.2.2 Event-Driven Architecture

Das Spiel nutzt C# Events für lose Kopplung zwischen Systemen. Beispiel: Wenn sich Player-Stats ändern, werden diese über Events kommuniziert:

```

1 public class PlayerStats : MonoBehaviour
2 {
3     public event Action OnHealthChanged;
4     public event Action OnLevelUp;
5     public event Action<int, int> OnLevelUpWithLevel;
6
7     public void TakeDamage(float damage)
8     {
9         health -= damage;
10        OnHealthChanged?.Invoke();
11    }
12 }
13
14 public class PlayerResourceBarsUI : MonoBehaviour
15 {
16     private void Start()
17     {
18         PlayerStats stats = player.GetComponent<PlayerStats>();

```

```

19     stats.OnHealthChanged += UpdateHealthBar;
20 }
21 }

```

Listing 6.2: Event-Driven Architecture Beispiel

Vorteile:

- Lose Kopplung zwischen Systems
- UI aktualisiert automatisch bei Stat-Änderungen
- Leicht erweiterbar für neue Listener

6.2.3 Inventory System (MVC Pattern)

Das Inventory System implementiert ein MVC-ähnliches Pattern:

- **Model** (Inventory.cs): 49-Slot-Datenstruktur, Item-Verwaltung
- **Controller** (PlayerInventory.cs): Geschäftslogik, Add/Remove Items
- **View** (InventorySlotUI.cs): UI-Darstellung, Drag-and-Drop

Das System nutzt Scriptable Objects für Item-Daten:

- Jedes Item hat eine ItemData ScriptableObject-Definition
- ID-basiertes Lookup ermöglicht Serialisierung (ScriptableObjects sind nicht serialisierbar)
- 49 Slots können stackable (potions) oder non-stackable (equipment) Items halten

6.2.4 Scene Management

Das Spiel implementiert ein Scene Transition System mit nahtlosen Übergängen:

1. Spieler betritt Trigger Zone (SceneTrigger.cs)
2. Loading Screen wird angezeigt (mit UI in unscaled Time)
3. Szene wird asynchron geladen
4. SpawnPoint-System teleportiert Spieler zu korrekter Position
5. Loading Screen faded aus

Die Spawn Point Architektur ermöglicht es, mehrere Einstiegspunkte pro Szene zu haben (z.B. unterschiedliche Türen/Portale).

6.2.5 Save/Load System

Das Spiel nutzt ein Hybrid Save-System:

- **PlayerPrefs:** Einfache Daten (Lautstärke, Einstellungen)
- **JSON Files:** Komplexe Daten (Inventory, Stats, Position)
- **SceneData:** World-State persistent über SaveManager

Das Hauptproblem war die Serialisierung von ScriptableObject-Referenzen. Lösung: ID-basiertes Item-Lookup.

6.3 Performance-Optimierungen

6.3.1 Culling-System

Das CullingOptimizer Script deaktiviert Renderer für Objekte außerhalb der Camera View:

- Distance-based Culling mit konfigurierbarem Radius
- Signifikanter FPS-Boost in großen Szenen
- Automatisches Re-enabling wenn Objekt in Nähe kommt

6.3.2 Animator Culling

AnimatorCulling Script reduziert Animation Updates für Off-screen Objects:

- Three Culling Modes: Always Animate, Cull Update Transforms, Cull Completely
- Massive Reduktion von CPU-Last bei vielen Enemies
- Spieler bemerkt keinen visuellen Unterschied

6.3.3 Dynamic Quality

PerformanceOptimizer überwacht FPS und reduziert Quality bei Bedarf:

- Targets: 60 FPS
- Bei Unterschreitung: Particle Effects reduzieren, Shadow Quality senken, etc.
- Dynamisches Adjustment ohne Game-Pause

7 Entwicklungsprozess

7.1 Planung und Projektmanagement

7.1.1 Entwicklungsphasen

Das Projekt wurde in 5 Phasen über ca. 6 Monate entwickelt:

Phase 1 (Oct 2025) : Core Mechanics

- Movement System, Basic Combat, Player Sprites
- UI Grundlagen (Health Bar, Inventory)

Phase 2 (Nov 2025) : Gegner-Verhalten & Raycasting

- Raycasting-basiertes Obstacle Avoidance implementieren
- State Machine für Enemies
- Combat Music System

Phase 3 (Dec 2025) : RPG Systems

- Inventory System mit Equipment
- Level-Progression und Stat-Scaling
- Save/Load System

Phase 4 (Jan 2026) : Polish & Audio

- Sound Design und Music Integration
- UI Polish, Feedback-Systeme
- Performance Optimizations

Phase 5 (Feb 2026) : Boss Encounter & Testing

- Boss Wave System implementieren
- Playtesting und Bug Fixing
- Final Polish und Documentation

7.1.2 Versionskontrolle

Das Projekt nutzte Git `nostalelike.github.io` mit GitHub für Versionskontrolle:

- **Main Branch:** Stabiler, produktionsreifer Code
- **Develop Branch:** Integrationsbranch für Features
- **Feature Branches:** Isolierte Feature-Entwicklung
- **80+ Commits:** Regelmäßiges Commitment mit beschreibenden Nachrichten

7.2 Meilensteine und Deliverables

- **Oct 2025:** First Playable Demo mit Basic Movement + Combat
- **Nov 2025:** Gegner-Verhalten mit Raycasting funktionsfähig
- **Dec 2025:** Inventory System functional und Save/Load arbeitet
- **Jan 2026:** Audio System integrated, Sound Effects komplett
- **Feb 2026:** Boss Encounter spielbar, Game Loop geschlossen

7.3 Testing und Quality Assurance

7.3.1 Testing-Strategien

1. **Manual Playtesting:** Team-Mitglieder spielen regelmäßig das Spiel
2. **Debug Logging:** Debug.Log Statements in kritischen Code-Pfaden
3. **Unity Gizmos:** Visuelle Debugging von Erkennungsreichweiten und Raycasts
4. **Automated Checks:** Compile-time Checks, Unity's Scene validation

7.3.2 Bug Tracking

Bugs wurden über GitHub Issues verfolgt, mit Labels nach Kategorie:

- **bug:** Tatsächliche Fehler
- **enhancement:** Verbesserungen vorhandener Features
- **gameplay:** Balancing und Design Issues
- **performance:** Optimierungs-Gelegenheiten

7.4 Herausforderungen und Lösungen

7.4.1 Challenge 1: Enemy Pathfinding

Problem Enemies würden in Wänden steckenbleiben, wenn der Spieler dahinter war. Dadurch entstand schlechtes Gameplay-Gefühl.

Ansätze

1. Erste Idee: A* Pathfinding – zu komplex, keine NavMesh vorhanden
2. Zweite Idee: Simple Avoid-Walls – funktioniert nicht gut mit komplexen Geometrien
3. Dritte Idee (implementiert): Multi-directional Raycasting

Lösung Raycasting-System mit 3 Rays (forward, left 30°, right 30°) mit dynamischer Ausweichlogik

Resultat Enemies navigieren natürlich um Hindernisse, ohne Complex Pathfinding

7.4.2 Challenge 2: Scene Persistence

Problem Spieler-Position wurde nach Scene-Load zurückgesetzt. Außerdem spawn mehrere Manager-Instanzen bei jedem Scene-Load.

Lösung

- SpawnPoint-System: Named spawn locations in jeder Szene
- SceneTransitionManager verwaltet target spawn point
- Singleton Pattern mit frühen Instance-Checks verhindert Duplikate

Resultat Nahtlose Scene-Übergänge, kein Gameobject-Spam

7.4.3 Challenge 3: Combat Feel

Problem Angriffe fühlten sich instant und unfair an. Spieler konnte nicht auf Gegner-Angriffe reagieren.

Lösung

- Attack Telegraph System: 0.4s Wind-up vor Schaden
- Smaller Hit Range (1.0 Units) vs Larger Attack Range (2.0 Units)
- Sound- und Visual Feedback

Resultat Skill-basiertes Combat mit Ausweich-Möglichkeiten

7.4.4 Challenge 4: Inventory Serialisierung

Problem ScriptableObject-Referenzen können nicht direkt JSON-serialisiert werden.

Lösung ID-basiertes Lookup System:

- Jedes Item hat eine eindeutige String-ID
- Save speichert nur Item-IDs und Mengen
- Load rekonstruiert Items via Lookup-Dictionary

Resultat Robustes Save/Load System ohne Data Corruption

7.4.5 Challenge 5: Defense Scaling

Problem Bei hohen Leveln wird lineare Defense Formel unausgeglichen. Spieler werden invincibel.

Lösung Diminishing Returns Formel:

$$\text{reduction} = \min\left(\frac{\text{defense}}{\text{defense} + 20}, 0.8\right) \quad (7.1)$$

Resultat Defense ist immer wertvoll, aber mit abnehmenden Erträgen. Max 80

7.5 Design-Entscheidungen

7.5.1 Top-Down vs. Side-Scroller

Wahl Top-Down Perspektive

Rationale • Besser für Area Exploration (NosTale Inspiration)

- Einfacher Collision Detection
- Camera Kontrolle ist straightforward

7.5.2 Combat System: Target Lock vs. Free Aim

Wahl Mouse-directed Free Aim mit Arc Detection

Rationale • Höhere Skill Expression

- Erlaubt Multi-Target Attacken
- Belohnt gute Positioning

7.5.3 Gegnersteuerung: Raycasting vs. A* Pathfinding

Wahl Raycasting-basiertes Obstacle Avoidance

Rationale • Keine NavMesh-Baking nötig

- Schneller, weniger CPU-intensiv
- Erfüllt Kurs-Anforderungen (Gegner-Verhalten Feature)
- Einfacher zu debuggen und visualisieren

Trade-offs • Nicht optimal für komplexe Multiroom-Navigation

- A* wäre für größere Dungeons besser
- Aber für dieses Projekt ausreichend

7.5.4 Save System: Local vs. Cloud

Wahl Local JSON Files

Rationale • Scope Limitation (keine Backend verfügbar)

- Schneller zu implementieren
- Keine Network-Latenz

Future Enhancement Cloud Save wäre möglich mit REST API

8 Fazit

8.1 Zusammenfassung der Ergebnisse

Das Fragments of Silvaris Projekt ist fertig. Das Spiel funktioniert und zeigt verschiedene Game Development Konzepte in der Praxis.

Alle Kern-Features sind implementiert und das Spiel erfüllt die Anforderungen der Spieleprogrammierung Vorlesung. Besonders das Raycasting-basierte System für Gegner-Verhalten zeigt, wie man mit cleveren Lösungen gute Ergebnisse erreichen kann.

Das Team bestand aus Aaron Seitz (3006614), Robin Fitzek (3006199), Burak Yigit (3005865) und Paul Weber (88487), die gemeinsam über einen Zeitraum von Oktober 2025 bis Februar 2026 an diesem Projekt gearbeitet haben.

8.2 Erreichte Ziele

- ✓ Ausgefeiltes Gegner-Verhalten mit Raycasting-basiertem Obstacle Avoidance
- ✓ Engaging Combat System mit Arc-Detection und Telegraph-Mechanik
- ✓ Komplettes RPG-System (Inventory, Equipment, Leveling)
- ✓ Dynamisches Audio-System mit State-based Music
- ✓ Save/Load Funktionalität mit persistenten Spielzuständen
- ✓ Boss Encounter mit Wave-basiertem Design
- ✓ Polierte Benutzeroberfläche mit vollständigem Feedback

8.3 Reflexion über Projektziele

8.3.1 Was gut lief

- **Raycasting-Implementation:** Übertraf Erwartungen an Eleganz und Effektivität
- **Team Kollaboration:** Git-Workflow war effektiv, Merge Conflicts minimal
- **Iterative Development:** Regelmäßiges Playtesting ermöglichte schnelle Verbesserungen
- **Code Organization:** Saubere Architektur mit Singleton und Event Patterns machte Erweiterungen einfach

8.3.2 Was verbessert werden könnte

- **Frühere Planung:** Noch detailliertere GDD zu Anfang hätte Rework reduziert
- **Frühere Tests:** Unit Tests für kritische Systeme hätten Bugs schneller gefunden
- **Dokumentation während Dev:** Laufende Dokumentation statt alles am Ende
- **Asset Pipeline:** Formalisierte Asset Import Settings hätten Qualität erhöht

8.4 Probleme und Lösungswege

Siehe Kapitel 7 Abschnitt 7.4 für detaillierte Behandlung der Herausforderungen und deren Lösungen. Die wichtigsten Takeaways:

- Einfache Lösungen sind oft besser als komplexe (Raycasting vs. A*)
- Frühe Iteration und Testing ist kritisch
- Gutes Feedback (Visual, Audio, Haptic) ist essentiell für Game Feel
- Balancing ist ein iterativer Prozess

8.5 Zukünftige Erweiterungsmöglichkeiten

8.5.1 Technische Enhancements

- **A* Pathfinding:** Implementierung zum Vergleich mit Raycasting
- **Multiplayer/Co-op:** Netzwerk-Architektur mit Photon oder Netcode
- **Mobile Support:** Touch Controls und Responsive UI
- **VR Support:** Spatial Audio und Immersive Controls

8.5.2 Content Expansionen

- **Enemy Varietät:** Mehr Enemy-Typen mit einzigartigen Verhaltensweisen
- **Mehr Biome:** Zusätzliche Welten mit unterschiedlichen Themes
- **Quest System:** Branching Narrative mit multilen Missions
- **Skill Tree:** Charakter-Customization durch Skill-Progressionen

8.5.3 Quality of Life

- **Cloud Save:** Cloud-basierte Speicherung mit Account-System
- **Rebindable Controls:** Vollständig customisierbare Key-Bindings
- **Accessibility:** Colorblind Modes, Difficulty Settings, Subtitles
- **Modding Support:** Lua-basierte Modding API

8.6 Persönliche Erkenntnisse und Learnings

8.6.1 Technische Insights

- Raycasting ist unglaublich vielseitig für Game Logic (nicht nur Graphics)
- Event-basierte Architektur skaliert besser als direkte Referenzen
- Performance ist ein Feature, keine Nachgedanke
- Code Organization ist Wert der sich schnell auszahlt

8.6.2 Game Design Insights

- Feel ist genauso wichtig wie Mechanik
- Player Feedback (Visual, Audio, Haptic) ist kritisch
- Balancing benötigt Data, nicht nur Intuition
- Iteration ist das wichtigste Tool des Game Designers

8.6.3 Team-Erkenntnisse

- Gute Kommunikation ist Foundation für erfolgreiches Teamwork
- Code Reviews verhindern Bugs und verbessern Code Quality
- Regelmäßiges Syncing verhindert Rework
- Klare Ownership von Komponenten ist wichtig

8.7 Schlusswort

Das Fragments of Silvaris Projekt durchlief den kompletten Game Development Cycle von der Idee bis zur fertigen Dokumentation. Das Spiel zeigt solide Software Engineering Praktiken, kreative Problemlösungen und ein gutes Verständnis von Game Development Konzepten.

Besonders das Raycasting-basierte System für Hinderniserkennung zeigt, dass gutes Design zu einfachen und effizienten Lösungen führt. Statt komplexem A* Pathfinding wurde eine einfachere Lösung gewählt, die weniger Rechenleistung braucht und leichter zu debuggen ist.

Das Team ist stolz auf die erreichten Ergebnisse und freut sich auf die Präsentation und das Feedback zur abgeschlossenen Arbeit.

9 Verwendete Assets und Lizenzen

Das Projekt nutzt verschiedene externe Assets für Grafik, Sound und Musik. Alle verwendeten Assets sind ordnungsgemäß lizenziert und die Lizenzdokumente befinden sich im Ordner `Assets-License/`.

9.1 Grafik-Assets

9.1.1 Pixel Art Potion Pack (Animated)

- **Autor:** Karsiori
- **URL:** <https://karsiori.itch.io/pixel-art-potion-pack-animated>
- **Beschreibung:** 150 animierte Tränke in 12 verschiedenen Typen
- **Lizenz:** Siehe `Assets-License/Pack Info.txt`

9.1.2 Farm RPG - Character and Environment Sprites

- **Autor:** EmanuelleDev
- **URL:** <https://emanuelledev.itch.io/farm-rpg>
- **Beschreibung:** 2D Sprite Pack für RPG Charaktere und Umgebungen
- **Lizenz:** Siehe `Assets-License/Documentation.txt`

9.1.3 Kittens & Cats Character 32x32 Pixel Animation Pack

- **Autor:** Last-Tick
- **URL:** <https://last-tick.itch.io/kittens-cats-character-32x32-pixel-animation-pack>
- **Beschreibung:** Animierte Katzen-Charakter Sprites für Pixel Art Spiele
- **Lizenz:** Siehe `Assets-License/Leohpaz Licensing.txt`

9.2 Audio-Assets

9.2.1 Musik

Fantasy RPG Soundtrack Music

- **Autor:** AlkaKrab
- **URL:** <https://alkakrab.itch.io/fantasy-rpg-soundtrack-music>
- **Beschreibung:** Hintergrundmusik für RPG-Spiele
- **Lizenz:** Siehe `Assets-License/AlkaKrab Music License Info.pdf`

9.2.2 Sound Effects

RPG Essentials Sound Effects Free

- **Autor:** Leohpaz
- **URL:** <https://assetstore.unity.com/packages/audio/sound-fx/rpg-essentials-sound-effects-free>
- **Beschreibung:** Kostenlose Sound Effects für RPG-Spiele vom Unity Asset Store
- **Lizenz:** Siehe `Assets-License/Leohpaz_Licensing.txt`

9.3 Lizenzdokumente

Alle Lizenzdokumente befinden sich im Ordner `Assets-License/` im Projekt-Repository und enthalten die vollständigen Nutzungsbedingungen für die verwendeten Assets:

- `Pack Info.txt` – Lizenz für Pixel Art Potion Pack
- `Documentation.txt` – Lizenz für Farm RPG Sprites
- `Leohpaz_Licensing.txt` – Lizenz für Sound Effects und Cat Sprites
- `AlkaKrab Music License Info.pdf` – Lizenz für Fantasy RPG Musik